SE LAB5

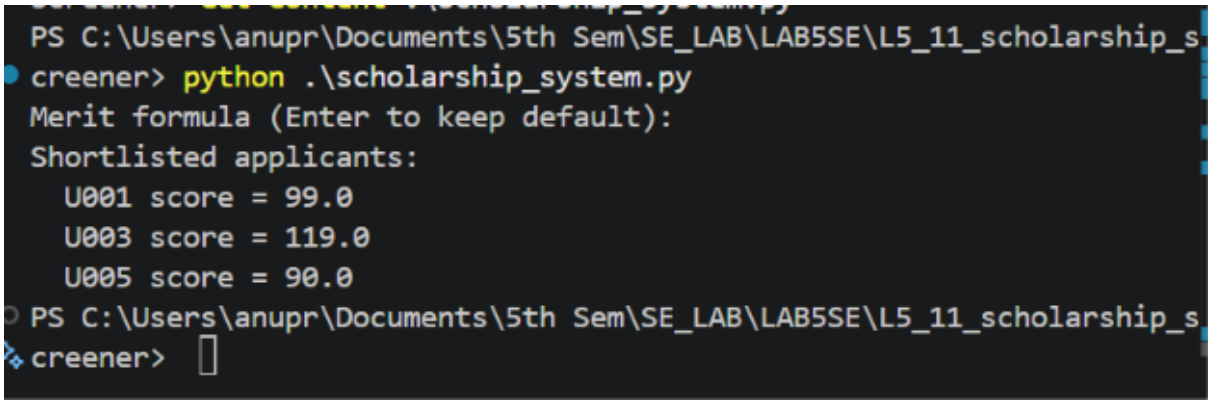
Static Code Analysis

Name: Anupriya

SRN: PES2UG24CS074

Section: 5B

Before Static Code Analysis



1. Issue identification and resolution table

Issue	Type	Line	Description	Fix approach
Unsafe eval()	Security	12	Bandit B307; medium severity. Evaluates a formula as Python code.	Replace with safe arithmetic that doesn't execute arbitrary code.
Mutable default argument	Bug	14	Pylint W0102; the logs=[] list can be shared between calls.	Use None and create a new list inside the function.
Bare except	Error handling	37	Pylint W0702, Flake8 E722, Bandit B110. Can hide unexpected failures.	Catch specific exceptions and handle them appropriately.
Unused imports	Code quality	2–3	Pylint W0611 and Flake8 F401 report unused math and sys.	Remove unused imports.

File handling	Code quality	29, 36	Files aren't managed with with, and encoding isn't specified.	Use with open(..., encoding="utf-8").
Boolean comparison	Style	43	Flake8 E712 and Pylint C0121. Compares a Boolean explicitly to True.	Use if results[r]["eligible"]:
Function naming	Style	40	Pylint C0103; printShortlist doesn't follow snake_case.	Rename it to print_shortlist().
Missing docstrings	Code quality	Multiple	Pylint reports missing module and function docstrings.	Add concise explanatory docstrings.

2. Static analysis results

Pylint:

10.00 / 10

No Pylint messages in the final run.

Flake8

No violations

The final command produced no output.

Bandit

0 issues identified

No high-, medium-, or low-severity issues in the final scan.

3. Reflection questions and answers

Q1. Why is static code analysis important?

Static code analysis helps identify programming errors, security vulnerabilities, and coding-standard violations without executing the program. It improves code quality, readability, maintainability, and security.

Q2. What is the difference between Pylint, Flake8, and Bandit?

- **Pylint:** Checks code quality, naming conventions, programming errors, and other potential problems.
- **Flake8:** Checks Python style and formatting rules, along with selected programming errors.
- **Bandit:** Examines Python code for common security issues, such as unsafe use of eval().

Q3. What were the major problems found in the original scholarship screening program?

The original program contained unsafe use of eval(), a mutable default argument, broad exception handling, file-handling issues, unused imports, and insufficient validation of applicant information.

Q4. How were the identified issues resolved?

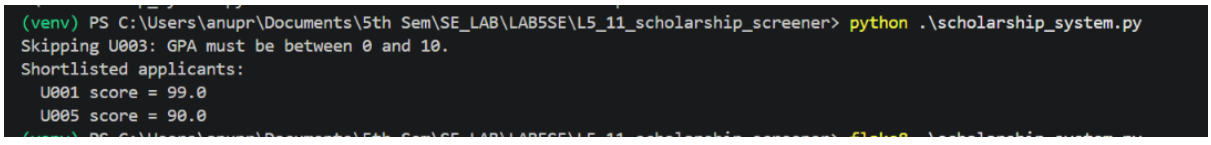
The code was improved by replacing eval() with a dedicated merit-score function, using None for the optional logging argument, catching specific exceptions, opening files with context managers and explicit encoding, removing unused imports, and validating GPA, credits, and income.

Q5. How did you verify the corrections?

I reran the program and all three static analysis tools. Pylint returned a score of 10.00/10, Flake8 reported no violations, and Bandit identified no issues. The execution test also confirmed that invalid applicant U003 was skipped.

Q6. What did you learn from this lab?

I learned that static analysis tools help detect different categories of code problems. Using multiple tools provides broader coverage than relying on a single tool. I also learned that validating inputs, handling exceptions properly, and following coding standards can improve the reliability and security of a Python application.



```
(venv) PS C:\Users\anupr\Documents\5th Sem\SE_LAB\LAB5SE\L5_11_scholarship_screener> pylint .\scholarship_system.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.60/10, +0.40)

(venv) PS C:\Users\anupr\Documents\5th Sem\SE_LAB\LAB5SE\L5_11_scholarship_screener> flake8 .\scholarship_system.py
(venv) PS C:\Users\anupr\Documents\5th Sem\SE_LAB\LAB5SE\L5_11_scholarship_screener> bandit .\scholarship_system.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.14.0
Run started:2026-10-14 08:44:52.864769+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 79
        Total lines skipped (#nosec): 0
```